Nega shu texnologiya

Loyihadagi har bir tanlov: bu nima o‘zi, nega kerak bo‘ldi, va o‘rniga nima bo‘lishi mumkin edi — nega u emas.

BM25 | Qdrant | e5 embedding | Gibrid qidiruv | Ball fusion | Chunking | RAG | JSON sxema | FastAPI | SQLite | Freymvork yo‘q | Transport

BM25

SO‘Z BO‘YICHA QIDIRUV

NIMA U

1994-yilda ishlab chiqilgan **reyting formulasi** (Okapi BM25) — kalit so‘z bo‘yicha qidiruvning sanoat standarti. Google ham, Elasticsearch ham shu g‘oyaga tayanadi. Uch qismdan iborat:

tf — so‘z hujjatda ko‘p uchrasa, hujjat tegishliroq. Lekin *to‘yinish* bilan: 10 marta uchrashi 1 martadan 10 barobar emas, ~2 barobar yaxshi.

idf — so‘z *kam* hujjatda uchrasa, ko‘proq ma‘lumot beradi. «kodeks» bizning 7 383 moddaning hammasida bor → qiymati deyarli nol. «o‘g‘rilik» 7 ta moddada bor → juda qimmatli.

Uzunlik — uzun hujjatda tabiiy ravishda ko‘proq so‘z bor; bu afzallik ayiriladi.

```
ball = Σ idf(so'z) · tf·(k1+1) / (tf + k1·(1 - b + b·dl/avgdl))

k1 = 1.5   to'yinishni boshqaradi
b  = 0.75  uzunlik normallashtirishini boshqaradi
```

Amalda: «o‘g‘rilik» so‘zli savolda sarlavhasi *O‘g‘rilik* bo‘lgan modda **12.5** ball, keyingisi **7.9**. Bu farq ishonchli — tasodif emas.

TANLANGAN

`BM25, o‘z implementatsiyasi · ~40 qator · xotirada`

Teskari indeks: savoldagi so‘z bo‘yicha faqat *o‘sha so‘z uchraydigan* moddalar ko‘riladi, 7 383 tasi emas. O‘lchangan: **0.5 ms**.

NEGA BOSHQASI EMAS

`Oddiy «so‘z bormi»` SINALDI	Loyihada avval shu bor edi va yiqildi. Qaysi so‘z muhimligini bilmaydi: «javobgarlik» har moddada bor va «o‘g‘rilik» bilan *teng vaznda* hisoblanardi, natijada umumiy moddalar aniq moddani bosib ketardi. BM25 ning `idf` qismi aynan shuni hal qiladi.
`Elasticsearch`	Butun servis: JVM, klaster, indeks boshqaruvi. 7 383 qisqa hujjat xotiraga sig‘adi, ya‘ni ES *imkoniyat* qo‘shmaydi — faqat o‘rnatish va boshqarish ishini qo‘shadi. Uning qiymati bizda yo‘q hajmda ochiladi.
`rank_bm25`	Formulani yozish kerak emas edi, lekin tokenizatsiyani kutubxona o‘z qo‘liga oladi. Bizga esa o‘zbek tiliga xos tokenizatsiya kerak: stopword, sinonim, va **kirill → lotin transliteratsiya** (`ўғрилик → o‘g‘rilik` , aks holda kirill savol hech nima topmaydi). Ya‘ni tokenizatsiyani baribir o‘zimiz yozardik — qolgani 40 qator.

Qdrant

VEKTOR BAZASI

> NIMA U
>
> Vektor bazasi 768 raqamli nuqtalarni saqlaydi va *«shu nuqtaga eng yaqin 40 tasini top»* degan savolga tez javob beradi. Oddiy baza buni qila olmaydi: u matn **tengligini** biladi, **yaqinligini** bilmaydi.
>
> Bizga uch narsa kerak edi: (a) vektor qidiruv, (b) qidiruvni payload bo‘yicha **filtrlash** — masalan faqat `code = mehnat_kodeksi` ichidan, (c) butun kolleksiyani **qaytib o‘qish** — BM25 indeksini qurish uchun.

> TANLANGAN
>
> `Qdrant · Docker · cosine · 7 383 nuqta`
>
> Uchala ehtiyoj ham uning asosiy imkoniyati. Docker’da bitta qatorda ko‘tariladi. O‘lchangan qidiruv vaqti: **35 ms**, mahalliy, tarmoqsiz.

NEGA BOSHQASI EMAS

`Chroma`	Soddaroq va o‘rnatish osonroq. Lekin kerak bo‘lgan uchta narsadan ikkitasi — filtrli qidiruv va butun kolleksiyani `scroll` — unda kuchsizroq.
`FAISS`	Juda tez, lekin u **kutubxona, baza emas**: payload saqlamaydi, filtr yo‘q, diskka saqlashni o‘zingiz qilasiz. Modda metadatasini alohida joyda saqlab ikkisini sinxron tutish kerak bo‘lardi — ortiqcha xato manbasi.
`pgvector`	**Haqiqiy alternativa — ishlagan bo‘lardi.** SQL filtri va vektor bir joyda. Lekin Postgres servisini ko‘tarish kerak, bizning relyatsion ehtiyoj esa bitta jadval (uni SQLite qoplaydi). Ya‘ni foyda bermasdan bitta servis qo‘shiladi.
`Pinecone / Weaviate`	Boshqarish kerak emas, lekin tashqi servis: API kaliti, har savolga tarmoq kutish, va ma‘lumot mashinadan chiqib ketadi.

multilingual-e5-base

EMBEDDING MODELI

NIMA U

Embedding — model matnni 768 ta raqamga aylantiradi. Bu raqamlar fazoda nuqta kabi, va *ma‘nosi o‘xshash* matnlar bir-biriga yaqin tushadi. O‘xshashlik ikki vektor orasidagi **burchak kosinusi** bilan o‘lchanadi: 1.0 — bir xil yo‘nalish, 0 — aloqasi yo‘q.

Shuning uchun «ishdan bo‘shatish» va «mehnat shartnomasini bekor qilish» yaqin tushadi — bitta ham umumiy so‘z bo‘lmasa ham.

TANLANGAN

`intfloat/multilingual-e5-base · 768 · mahalliy CPU`

Qidiruv uchun maxsus o‘qitilgan, o‘zbek va rus tilini qoplaydi, API kaliti kerak emas va pul ketmaydi. Savolni vektorga aylantirish: **153 ms**.

Nozik joy: e5 ikki prefiksni talab qiladi — indekslanadigan matn `passage:` , savol `query:` . Aralashtirilsa aniqlik tushadi, shuning uchun ikkisi bitta faylda (`embedding.py`) turadi.

NEGA BOSHQASI EMAS

`MiniLM` SINALDI	Kichikroq va tezroq, lekin **yiqildi**: har nomzodga 0.69–0.71 ball berib, hech narsani farqlamagan va to‘g‘ri moddani butunlay o‘tkazib yuborgan. Yuridik matnda moddalar bir xil lug‘atni ishlatadi — MiniLM bu farqni sezmaydi.
`OpenAI text-embedding-3`	Sifat yuqori bo‘lishi mumkin, lekin har savolga API chaqiruvi: pul va tarmoq kutish. 7 383 moddani indekslash ham pullik. Va kalit bo‘lmasa *butun qidiruv* ishlamay qoladi — hozir kalitsiz ham ishlaydi.
`bge-m3`	Yana aniqroq bo‘lishi mumkin, lekin ~2 GB yuklab olish va CPU da sezilarli sekinroq. e5-base allaqachon 0.978 Hit@5 beryapti.

Ikki qidiruv birga

GIBRID QIDIRUV

NEGA IKKISI

Ularning kuchli va kuchsiz tomoni **aynan teskari**:

```
savol: «ishdan bo‘shatishsa nima qilaman»
  modda matni: «mehnat shartnomasini bekor qilish»
  dense  → TOPADI    (ma'no yaqin)
  BM25   → topmaydi   (bitta ham umumiy so'z yo'q)

savol: «o‘g‘irlik uchun jazo»
  modda sarlavhasi: «O‘g‘rilik»
  dense  → ko'madi    (cosine'lar 0.79–0.81 da zich)
  BM25   → TOPADI     (12.5 va 7.9)
```

```
BM25   → TOPADI     (12.5 vs 7.9)
```

Ya‘ni bittasi yetmaydi. Ikkisi bir-birining ko‘r nuqtasini yopadi.

TANLANGAN

`dense + BM25 + kodeks doirasidagi dense`

Har savolda uch manbadan 40 ta nomzod, keyin ballar birlashtiriladi. Natija: uch tilda 5/5 to‘g‘ri modda birinchi o‘rinda.

NEGA BOSHQASI EMAS

`Faqat dense`	Bu korpusda cosine‘lar 0.79–0.81 da zich, ya‘ni tartib deyarli tasodif. «o‘g‘rilik» savoli topilmaydi.
`Faqat BM25`	Parafrazani umuman topmaydi. Spec esa «noaniq yoki og‘zaki ifodalangan» savollar kategoriyasini talab qiladi.

Ball bo‘yicha fusion

IKKI NATIJANI QANDAY QO‘SHISH — LOYIHADAGI ENG MUHIM QAROR

MUAMMO

Dense cosine beradi (0.79–0.81), BM25 esa butunlay boshqa shkalada ball beradi (0–15). Ikkisini qanday qo‘shish kerak?

Standart javob — **RRF**: ball emas, *o‘rin* qo‘shiladi. Bu korpusda noto‘g‘ri bo‘lib chiqdi.

Chunki yuridik moddalar bir xil lug‘atni ishlatadi, shuning uchun e5 o‘xshashliklari zichlashadi — 40 nomzod 0.79–0.81 orasida, ya‘ni dense *tartibi* deyarli tanga tashlash. BM25 farqi esa keng va ma‘noli: 12.5 va 7.9.

RRF ball kattaligini tashlab yuboradi — **aynan shu yerda farqlaydigan signalni**. Natijada *o‘g‘rilik* savolining to‘g‘ri moddasi (Jinoyat kodeksi 169-modda) 5-o‘rinda ko‘milib qolgan.

TANLANGAN

`absolyut ball fusion`

```
fused = 1.0 · dense + 0.8 · leksik + 0.3 · kodeks

dense  = clamp((cosine - 0.75) / 0.20, 0, 1)   absolyut, nisbiy emas
leksik = bm25 / (bm25 + 6)                     to'yinuvchi
kodeks = 0.3 agar savol shu kodeksni atagan    tekis
```

dense absolyut: nomzodlar to‘dasiga nisbatan normallashtirilsa, bir-biriga o‘xshash o‘rtamiyona nomzodlar to‘dasi o‘zining eng yaxshisini 1.0 ga ko‘taradi — ya‘ni hech narsa topilmaganda ham «ishonchli» ball chiqadi.

leksik to‘yinuvchi: kuchli aniq moslik (12.5) tasodifiy so‘z ustma-ustligidan (2.0) ancha ko‘p turishi kerak, lekin cheksiz o‘smasligi ham kerak.

kodeks tekis: avval bu dense ballga *qo‘shimcha* hissa qo‘shardi, ya‘ni kodeks aniqlanganda dense vazni jimgina 1.0 dan 1.6 ga chiqardi — va cosine 0.845 bo‘lgan umumiy modda hal qiluvchi so‘z sarlavhasida turgan moddani bosib ketardi. Tekis bo‘lsa, bitta kodeks ichida o‘zaro bekor bo‘ladi.

Natija: 169-modda 5-o‘rindan **1-o‘ringa** chiqdi.

NEGA BOSHQASI EMAS

RRF SINALDI	Standart usul va ball shkalasini bilish kerak emas — lekin aynan shu korpusda ball farqi ma‘noli, o‘rin esa tasodif. To‘g‘ri moddani ko‘mib tashladi.
`Cross-encoder reranker`	Eng aniq usul, lekin `bge-reranker-v2-m3` ~2.2 GB yuklab olish va CPU da har savolga qo‘shimcha 1–3 sekund. Ball fusion allaqachon 5/5 beryapti.
`LLM bilan reranking`	Yuklab olish kerak emas, lekin har savolga *qo‘shimcha* LLM chaqiruvi — vaqt va narx ikki barobar. Vaqtning 98.7% i allaqachon LLM kutishga ketyapti.

Modda bo‘yicha chunking

MATNNI QANDAY BO‘LAKLASH

NIMA U

Chunking — uzun matnni qidiruv uchun bo‘laklarga bo‘lish. Har bo‘lak alohida vektorga aylanadi va qidiruv natijasida *bo‘lak* qaytadi.

Ya‘ni bo‘lak chegarasi javobning aniqligini belgilaydi: bo‘lak «qaysi moddadan?» degan savolga javob bermasa, sitata ham bermaydi.

TANLANGAN

`bitta bo‘lak = bitta modda`

`citations[].article` konstruksiya bo‘yicha aniq, taxmin emas. Va o‘zbek qonunchiligi allaqachon moddalarga bo‘lingan — ya‘ni bu tabiiy chegara, biz o‘ylab topgan narsa emas.

NEGA BOSHQASI EMAS

`1200 belgi bo‘yicha` SINALDI	Universal usul, loyihada avval shu bor edi. Bo‘lak ikki modda o‘rtasidan kesib o‘tishi mumkin, ya‘ni «bu matn qaysi moddadan?» degan savolga javob yo‘q. Spec esa har sitatada `article` va `part` ni **majburiy** qilgan. O‘lchangan natija: shunday 60 bo‘lak `lex:#art-` degan bo‘sh sitata bergan.
`Sliding window`	Kontekst yo‘qolmaydi, lekin bitta modda bir necha bo‘lakka tushadi: `chunk_id` barqaror emas va bitta modda qidiruvda bir necha marta chiqadi.

RAG

NEGA UMUMAN SHU YONDASHUV

NIMA U

Retrieval-Augmented Generation — savol kelganda avval bazadan tegishli matn *topiladi*, keyin model *faqat shu matndan* javob yozadi. Model qonunni «biladi» deb taxmin qilinmaydi: qonun unga har safar berib turiladi.

TANLANGAN

`RAG + sitata tekshiruvi`

Qonun **ma‘lumot** bo‘lib qoladi, model vazniga singdirilmaydi. Shuning uchun manbani ko‘rsatish mumkin, qonun o‘zgarsa indeksni qayta qurish yetarli, va modelning har bir da‘vosini manbaga qarshi tekshirish mumkin.

NEGA BOSHQASI EMAS

`Fine-tuning`	Qonun model vaznlariga «singdirilardi», lekin keyin **manbani ko‘rsatib bo‘lmaydi** — javob qaysi moddadan kelganini aytolmaydi. Qonun o‘zgarsa qayta o‘qitish kerak. Spec esa har javobda modda va harfma-harf parcha talab qiladi.
`To‘g‘ridan modelga savol`	Eng oddiy, lekin model o‘ylab topadi: ishonchli ovozda «25 kun» deyishi mumkin. Yuridik savolda bu xavfli va tekshirib bo‘lmaydi.
`Butun kodeksni promptga`	7 383 modda — millionlab token. Hech qaysi model kontekstiga sig‘maydi, va har savolga shuncha token to‘lash mumkin emas.

Strukturalangan JSON

MODEL JAVOBINING SHAKLI

NIMA U

Modelga javob shaklini **JSON sxema** bilan majburlash. Provayder javobni server tomonda tekshiradi, ya‘ni shakli buzilgan javob kelmaydi.

Model qaytaradigan narsa: javob matni (`[1]` belgilari bilan), va har bir belgi uchun qaysi kontekstdan olgani plus **harfma-harf ko‘chirgan parcha**.

TANLANGAN

`JSON schema · majburlangan`

Chunki keyingi qadam — **tekshiruv**: ko‘chirgan gap modda matnida aynan bormi? Buni mashina tekshirishi uchun da‘vo mashina o‘qiydigan shaklda bo‘lishi kerak.

NEGA BOSHQASI EMAS

`Erkin matn + regex`	«Bu gap qaysi moddadan?» degan savolga prozadan javob ajratish ishonchsiz —

	model har safar boshqacha yozadi.
`Promptda «sitata ham yoz»`	Model aytadi, lekin shakli har xil bo‘ladi va ba‘zan umuman yozmaydi. Sxema majburlansa, maydon *bor* bo‘lishi kafolatlanadi.
`Function calling`	Mohiyatan bir xil natija, lekin `response_format` / `output_config.format` to‘g‘ridan javob shakli uchun mo‘ljallangan va soddaroq.

FastAPI + pydantic

HTTP QATLAMI

> NIMA U
>
> Python veb-freymvorki. So‘rov shaklini Python tiplari bilan yozasiz, validatsiya **avtomat** bo‘ladi, va `/docs` da OpenAPI hujjati o‘zi paydo bo‘ladi.

> TANLANGAN
>
> `FastAPI + pydantic + uvicorn`
>
> Spec aniq 400 holatlarini talab qiladi: 11 savol, takroriy `id`, bo‘sh savol, 2000+ belgi. pydantic bilan bu *sxemaning o‘zi* — alohida validatsiya kodi yo‘q. Va rad etish hech qanday vektor hisoblanmasdan, pul ketmasdan bo‘ladi.

NEGA BOSHQASI EMAS

`Flask`	Validatsiyani qo‘lda yozish kerak — har bir 400 holati uchun alohida kod va alohida xato.
`Django`	To‘liq freymvork: ORM, admin, template. Bizga hech biri kerak emas — 5% ini ishlatardik.
`Yalang’och ASGI`	Marshrutlash, validatsiya, xato konvertlari — hammasi qo‘lda.

SQLite

BATCH NATIJALARINI SAQLASH

> NIMA KERAK EDI
>
> Uch narsa: `batch_id` ni **band qilish** (ikki bir vaqtda kelgan so‘rov ikkalasi ham hisoblashni boshlamasligi uchun — ya‘ni *tranzaksiya*), natijani **30 kun** saqlash (spec talabi), va fon oqimi yozayotganda `GET` o‘qiy olishi.

> TANLANGAN
>
> `sqlite3 (Python ichida) · WAL rejimi`
>
> O‘rnatish kerak emas, tranzaksiya bor (`BEGIN IMMEDIATE` — band qilish poygasi uchun), WAL rejimi bir vaqtda o‘qish va yozishni beradi. Ehtiyoj — bitta jadval.

NEGA BOSHQASI EMAS

`Xotiradagi dict`	Server qayta ko‘tarilganda yo‘qoladi. Spec 30 kun saqlashni talab qiladi.
`Redis`	Bitta servis qo‘shiladi, va bizga kerak bo‘lgani — doimiylik va tranzaksiya — Redis ning kuchsiz tomoni.
`PostgreSQL`	Ishlagan bo‘lardi, lekin yana bitta servis boshqarish uchun. Ehtiyoj esa bitta jadval, bitta tranzaksiyali `INSERT` .

Freymvork ishlatilmagan

NEGA LANGCHAIN YOKI LLAMAINDEX EMAS

> NIMA U
>
> RAG qurish uchun tayyor freymvorklar: retriever, chunker, prompt shablonlari, vektor baza ulagichlari — hammasi bir paketda. Prototipni tez qurish uchun yaxshi.

> TANLANGAN
>
> `freymvork yo‘q — to‘g‘ridan kutubxonalar`
>
> Chunki bu loyiha aynan ikki narsaga tayanadi, va ikkisi ham freymvork abstraksiyasiga sig‘maydi:
>
> **1. Ball fusion formulasi.** RRF muammosini topish uchun xom cosine va BM25 ballarini *ko‘rish* kerak bo‘ldi. Freymvork ularni yashiradi — ya‘ni muammo topilmasdan qolardi.
>
> **2. Sitata tekshiruv darvozasi.** Model ko‘chirgan gapni manbaga qarshi solishtirish. Bu standart RAG qadami emas, o‘z mantiq.
>
> Yana bir sabab: freymvork versiyasi yangilanganda qidiruv xatti-harakati jimgina o‘zgarishi mumkin. Bizda esa har javobda `pipeline_version` bor va spec run‘larni taqqoslashni talab qiladi — ya‘ni xatti-harakat o‘zgarishi **ko‘rinishi** shart.

Ikki xil transport

SDK VA ODDIY HTTP

> MUAMMO
>
> Provayder qaysi bo‘lishini oldindan bilmaymiz — o‘qituvchi Claude, OpenAI, Gemini, DeepSeek yoki boshqasini qo‘yishi mumkin. Har biriga alohida SDK o‘rnatish — 11 ta bog‘liqlik.

> TANLANGAN
>
> `anthropic SDK + httpx`
>
> **Claude** → rasmiy SDK, chunki uning structured output shakli, auth va xato tiplari o‘z kodida.
>
> **Gemini** → `httpx` + REST, chunki u o‘z protokolida gaplashadi (`responseSchema` , `systemInstruction`) va bitta endpoint uchun katta SDK ortiqcha.
>
> **Qolgan 11 provayder** (OpenAI, Groq, DeepSeek, OpenRouter, Together, Mistral,

> xAI, Fireworks, Cerebras, Ollama, LM Studio) → `httpx` + **bitta funksiya**, chunki hammasi bir xil OpenAI chat-completions protokolida gaplashadi. Yangi provayder qo‘shish = jadvalga bitta qator, kod emas.

Tanlovlar tezlikka qanday ta‘sir qilgan

O‘LCHANGAN, TAXMIN EMAS

```
savolni vektorga aylantirish (e5, CPU)   153 ms
Qdrant vektor qidiruv                     35 ms
BM25 qidiruv (xotirada)                  0.5 ms
                                        ──────
jami mahalliy ish                        272 ms

LLM API kutish                        21 347 ms   ← vaqtning 98.7%
```

Ya‘ni mahalliy texnologiya tanlovi umumiy tezlikka deyarli ta‘sir qilmaydi — tezlik **provayder kalitiga** bog‘liq. GPU qo‘shish umumiy vaqtni ~0.6% yaxshilaydi.

GPU foyda beradigan yagona joy — indeksni qurish: o‘lchangani ~690 ms har modda, ya‘ni 7 383 modda uchun ~85 daqiqa CPU da. GPU bilan bir necha daqiqa bo‘lardi. Lekin bu **bir martalik**: server qayta ko‘tarilganda vektorlar qayta hisoblanmaydi, faqat payload o‘qiladi (~3 sekund).

Sinalgan va voz kechilgan: MiniLM embedding, RRF rank fusion, oddiy so‘z qidiruvi, belgi bo‘yicha chunking. To‘rtalasi ham *ishlatilgan* va o‘lchov natijasida almashtirilgan — nazariy emas.